

MLOps Engineer

CompTIA SecAI+: AI Security Certification Complete Exam Prep 2026 | Section 23: AI

1. MLOps: Operational Discipline for Machine Learning

Software engineering learned in the 2000s that building software and operating it reliably at scale are different disciplines — DevOps emerged to close that gap. Machine learning is now learning the same lesson. Building a model in a notebook and running AI systems reliably, securely, and in compliance with organizational policy at production scale require fundamentally different skills. MLOps Engineers are the practitioners who apply operational discipline to machine learning: they build the pipelines, platforms, and processes that take models from experimental notebooks to governed production deployments and keep them running trustworthily throughout their operational lifetime.

For SecAI+ candidates, MLOps is critically important because it is where many AI security controls are implemented in practice — and where many AI security failures originate. A model that was rigorously evaluated and signed off during development can be silently replaced, poisoned, or degraded if the operational pipeline lacks adequate security controls. MLOps Engineers are the practitioners responsible for ensuring that does not happen.

2. The MLOps Lifecycle: Where Security Must Be Embedded

MLOps covers the full lifecycle of an AI system in production. Security must be embedded at each stage rather than applied at deployment as a final check — by that point, a compromised artifact or poisoned dataset may already have passed through every evaluation gate.

MLOps Lifecycle Stage	Description	Key Security Concern
Experiment tracking	Capturing training run metadata: dataset version, code version, hyperparameters, metrics	Auditability — can behavior be traced back to a specific run?
Dataset management	Versioned, cataloged storage of training data with provenance metadata	Data integrity — has the dataset been tampered with since validation?
Model packaging and serving	Containerizing models and exposing them through APIs	Supply chain — is the artifact that is deployed the one that was evaluated?
Monitoring	Detecting performance degradation, data drift, concept drift	Adversarial detection — can drift patterns indicate active attack?
Automated retraining	Triggering pipeline re-runs when monitoring detects degradation	Hijack risk — can an adversary force a retraining cycle with poisoned data?
Model retirement	Decommissioning models no longer fit for purpose	Governance — are deprecated models fully removed from all serving paths?

3. Software CI/CD vs. ML CI/CD: Critical Differences

MLOps Engineers extend CI/CD principles to machine learning workloads, but the analogy is imperfect in ways that matter for security. In software CI/CD, a passing test suite is strong evidence that the software is correct and safe to deploy. In ML CI/CD, a passing evaluation on a standard test dataset is necessary but not sufficient — models can pass standard evaluation while being brittle against adversarial inputs, distribution shift, or specific demographic subgroups.

- **Standard test evaluation:** measures accuracy, precision, recall on a held-out test set drawn from the same distribution as training data. Passes cleanly for models with learned backdoors if the trigger is not in the test set.
- **Adversarial test evaluation:** tests model behavior on inputs crafted to cause misclassification. Required to detect backdoored models and evaluate robustness against adversarial input attacks.
- **Fairness metric evaluation:** tests model performance disaggregated by demographic subgroups against defined thresholds. Required for high-stakes AI systems before deployment.
- **Performance benchmarks:** tests latency, throughput, and memory footprint against service-level requirements. A model that meets accuracy targets but cannot serve requests within latency SLAs should not be deployed.
- **Security tests:** tests input validation, output filtering, and behavior under malformed inputs. Required to catch injection vulnerabilities in LLM-based systems before deployment.

Key distinction: A standard evaluation passing on a backdoored model is not a false negative from the test suite — it is the correct result for a test suite that does not include adversarial evaluation. ML CI/CD pipelines that omit adversarial test sets provide a false assurance of safety.

4. Model Registry as a Security Control Point

The model registry — a cataloged store of trained model artifacts with associated metadata — is one of the most important security controls in an MLOps architecture. It is the gate between training and production: only models that have been validated, reviewed, and approved should be registerable, and only registered models should be deployable.

- **Artifact integrity:** model files should be digitally signed before registration. An artifact registered without a signature provides no guarantee that the file was produced by the legitimate training pipeline rather than substituted after the fact.
- **Access controls:** only authorized pipeline service accounts should be permitted to register new model versions. A registry that allows any authenticated user to register artifacts provides much weaker guarantees.
- **Deployment lineage:** every production deployment should be traceable to a specific registry entry, which in turn is traceable to a specific training run, dataset version, and code version. This lineage is essential for incident response.
- **Rollback capability:** by maintaining references to previous production models, the registry enables fast rollback when a deployed model's behavior is found to be problematic — whether due to performance degradation or a detected adversarial modification.

- **SecAI+ connection:** Model registry practices map to AI supply chain security principles. A registry that enforces signed artifacts and pipeline-only registration is implementing supply chain integrity controls equivalent to software artifact signing in DevSecOps.

5. Real-World Incident: Model Registry Poisoning via Pickle Payloads (2024)

Real-World Incident — Protect AI Research on Hugging Face Hub (2024): In 2024, the Protect AI security research team published findings demonstrating that ML model repositories — including the widely used Hugging Face Hub — could be used to distribute malicious model files. A crafted model in PyTorch pickle format can execute arbitrary code when loaded, because PyTorch's native serialization format uses Python's pickle module, which executes deserialization code without restriction during model loading. An attacker who publishes a model that appears to be a legitimate open-source checkpoint but contains a pickle payload can compromise any system that loads the model without validation. The attack does not require any access to the target system beyond the ability to publish models to the repository.

The required defenses are: (1) use the SafeTensors format rather than pickle-based .pt or .bin files for model artifact storage and distribution — SafeTensors was designed specifically to prevent pickle-based arbitrary code execution; (2) enforce SafeTensors format as a model registry admission policy, rejecting pickle-format artifacts at registration time; (3) verify model artifact checksums and signatures before loading any external model. MLOps Engineers implementing model registries should treat this as a mandatory configuration baseline, not an optional hardening measure.

6. Automated Retraining: Risks Without Human Oversight Guardrails

Automated retraining — triggering model retraining when monitoring detects performance degradation — is a powerful MLOps capability that introduces specific security risks if implemented without adequate controls. An adversary who can influence the data pipeline feeding the retraining trigger can force a retraining cycle at a time of their choosing, potentially causing a poisoned model to be trained and deployed automatically — with no human in the loop to detect the anomaly.

- **Human approval gates:** production model promotion should require human approval even if training runs automatically. A human reviewing promotion metrics has the opportunity to detect anomalous evaluation results that automated gates might pass.
- **Retraining frequency monitoring:** unexpected spikes in retraining frequency — the pipeline retraining far more often than baseline — may indicate that an adversary is injecting data to repeatedly trigger the retraining threshold.
- **Adversarial test set evaluation:** retrained models should be evaluated against a held-out adversarial test set before promotion, specifically to detect whether the retraining incorporated a poisoning attack.
- **Anomaly detection on triggering data:** monitoring systems should flag when the data triggering a retraining event is itself anomalous — unusual distribution, unexpected source, or statistical properties inconsistent with the historical training data population.
- **Regulatory connection:** Fully automated, human-free model deployment pipelines optimize operational efficiency at the cost of human oversight. The EU AI Act (2024) and NIST AI RMF 1.0 both

require that decisions to eliminate human oversight from consequential AI operations be explicitly documented and risk-assessed.

7. Data and Model Drift as Security Signals

MLOps Engineers implement drift monitoring to detect when model inputs or outputs are changing in ways that suggest the model's performance is degrading. The security insight — often missed in purely operational MLOps implementations — is that specific drift patterns can be indicators of active adversarial attack rather than natural distribution shift.

Drift Type	Operational Meaning	Potential Security Signal
Data drift (input)	Input feature distributions are shifting from training distribution	Adversarial probing: systematic queries mapping model decision boundaries
Concept drift	Relationship between inputs and correct outputs has changed	May indicate the deployment environment has fundamentally changed or been manipulated
Prediction drift	Model output distribution is shifting without corresponding input drift	Possible backdoor activation: specific input patterns triggering hidden behavior
Sharp spike in specific inputs	Sudden high volume of inputs with similar feature values	Model extraction attack: volume querying to reconstruct model behavior
Gradual input distribution shift	Slow, systematic change in input features over weeks	Potential upstream data poisoning affecting production feature pipeline

MLOps Engineers who integrate security analysis into drift monitoring — routing anomalous drift patterns to the security team rather than treating all drift purely as a retraining trigger — provide early warning capability that security teams can act on. This maps directly to NIST CSF 2.0's DE.AE (Adverse Event Analysis) function.

8. Red Team Perspective: MLOps Pipeline as an Attack Surface

Red teams targeting MLOps infrastructure focus on the pipeline itself rather than the deployed model. Pipeline-level compromises are attractive because a successful attack can affect every model that goes through the pipeline — not just a single deployed system.

- Pipeline orchestration interfaces: Airflow web UI, Kubeflow Dashboard, and Prefect UI are frequently accessible without strong authentication. An attacker with access to these interfaces can modify pipeline DAGs, inject steps, or redirect artifact destinations.
- Pipeline configuration injection: places where pipeline parameters can be influenced externally — environment variables read from non-validated sources, configuration files pulled from external URLs — enable an attacker to change training behavior without modifying pipeline code.
- Weak model registry access controls: model registries that accept artifact uploads from any authenticated user rather than only from validated pipeline service accounts allow an attacker to insert a malicious model without running the training pipeline.

- Metric store tampering: monitoring systems fed false performance metrics can be made to suppress alerts about model degradation or suppress retraining triggers, leaving a degraded or poisoned model in production longer than intended.
- **Least-privilege pipeline service accounts:** Pipeline service accounts that can read all data stores and write to all registries provide an attacker who gains pipeline access with broad lateral movement capability. MLOps Engineers must apply least-privilege principles to pipeline service accounts with the same rigor applied to human user accounts.

9. Experiment Tracking for Reproducibility and Auditability

Experiment tracking captures the full provenance of every training run: the specific dataset version used (referenced by hash or version identifier), the code version (Git commit SHA), the hyperparameter configuration, the evaluation metrics, and the resulting model artifact. Tools such as MLflow Tracking, Weights and Biases, and Neptune.ai provide experiment tracking as a managed service or self-hosted platform.

The security value of experiment tracking is auditability: when a deployed model produces unexpected behavior — incorrect outputs, signs of a backdoor trigger, demographic bias discovered post-deployment — the audit trail must be traceable back through the specific training run, dataset version, and code version that produced it. ISO/IEC 42001:2023 (AI Management System) requires organizations to maintain records sufficient to enable AI system behavior to be explained and audited. Experiment tracking systems that are optional, inconsistently used, or disconnected from the deployment pipeline fail to provide this auditability.

- Dataset version tracking: record the specific dataset version hash or URI used for each training run — not just the dataset name. Two runs using 'training_data_v3' may have used different actual data if the underlying files were modified.
- Code version tracking: record the Git commit SHA of the training code, not just the branch name. Branch tips change; commit SHAs are immutable references.
- Artifact linkage: training artifacts (model weights, preprocessing artifacts, feature scalers) stored in the model registry should be linked to the experiment tracking entry that produced them, creating a bidirectional traceability chain.

10. Feature Store Operations and Security

Feature stores are centralized systems that compute, store, and serve features consistently across training and inference — ensuring that the features used in production inference match those computed during training. MLOps Engineers responsible for feature store operations must address several security concerns that go beyond operational health monitoring.

- Feature freshness and integrity: features must be updated on schedule and from trusted data sources. A feature store consuming data from a compromised upstream source will deliver poisoned features to both retraining pipelines and production inference.
- Feature access auditing: logging which models read which features creates an audit trail that supports both governance (which teams are using which data) and incident response (which inference endpoints received potentially compromised features).

- Feature versioning: changes to feature computation logic must be versioned and must not silently affect running models. A silent change to how a credit score feature is computed can change model behavior without any model retraining occurring.
- **MITRE ATLAS connection:** MITRE ATLAS v2.1 documents feature-level attacks where adversaries manipulate the feature computation process to influence model behavior. Feature store monitoring is the primary detection mechanism for this class of attack, and feature store access controls are the primary prevention mechanism.

Analogy: Think of the MLOps pipeline as an assembly line for AI systems. Quality control (evaluation gates, adversarial tests, human approval) catches defective products before they ship. But if an adversary can tamper with the assembly line itself — injecting steps, replacing materials, falsifying quality reports — every product that passes through is potentially compromised. Securing the pipeline is securing the assembly line, not just the product.

Key Terms Glossary

Term	Definition
MLOps	Machine Learning Operations — the discipline of applying operational engineering practices (automation, monitoring, versioning, governance) to the ML lifecycle to enable reliable, scalable, and governed production AI systems
Model registry	A cataloged store of trained model artifacts with metadata linking each artifact to its training run, dataset version, evaluation results, and deployment history. Serves as the gate between training and production deployment
Data drift	A statistical shift in the distribution of model input features away from the training distribution. Can indicate natural environmental change or adversarial probing of model decision boundaries
Concept drift	A change in the underlying relationship between model inputs and the correct output labels. Distinct from data drift — the input distribution may be stable while the correct outputs for those inputs have changed
SafeTensors	A model serialization format designed by Hugging Face to prevent arbitrary code execution during model loading by avoiding Python's pickle module. Preferred over .pt or .bin (pickle-based) formats for security
MLflow	Open-source MLOps platform providing experiment tracking, model registry, model serving, and project packaging capabilities. Widely adopted across industries for ML lifecycle management
Weights and Biases (W&B)	Commercial experiment tracking and model registry platform providing experiment visualization, artifact versioning, and collaboration tools for ML teams
Kubeflow Pipelines	Kubernetes-native ML pipeline orchestration platform. Its dashboard interface is a common attack target in MLOps red team exercises if accessible without strong authentication

Apache Airflow	General-purpose workflow orchestration platform widely used for ML data pipelines. The Airflow web UI requires strong authentication and network access controls to prevent pipeline DAG manipulation
Feature store	A centralized system that computes, stores, and serves features consistently across training and inference environments, preventing training-serving skew and providing a single auditable source for feature values
NIST AI RMF 1.0 GOVERN function	The first function in the NIST AI Risk Management Framework, covering establishing policies, processes, accountability structures, and human oversight requirements for AI risk management
ISO/IEC 42001:2023	International standard for AI Management Systems — requires organizations to establish governance structures, maintain auditability records, and implement continuous improvement processes for AI systems

Quick Revision Checklist

- MLOps Engineers build and operate the pipelines, platforms, and processes that keep AI systems running reliably, securely, and in compliance throughout their operational lifetime.
- ML CI/CD differs from software CI/CD: a passing standard evaluation does not detect backdoored models — adversarial test sets are required.
- The model registry is a security control point: enforce signed artifacts, pipeline-only registration, and deployment lineage tracing.
- SafeTensors format is required for safe model artifact storage and distribution — PyTorch pickle-format files (.pt, .bin) can execute arbitrary code on load.
- The 2024 Protect AI research demonstrated that malicious models distributed via Hugging Face Hub as pickle-format files can compromise any system that loads them without validation.
- Automated retraining without human approval gates enables an adversary who can influence training data to force a poisoning attack through the pipeline automatically.
- Drift monitoring serves dual purposes: operational (retraining trigger) and security (adversarial attack detection). Route anomalous drift patterns to the security team.
- MLOps pipeline orchestration interfaces (Airflow UI, Kubeflow Dashboard) are high-value attack targets — restrict to internal networks and require strong authentication.
- Feature store monitoring is the primary detection mechanism for MITRE ATLAS-documented feature-level attacks.
- Experiment tracking provides the auditability chain required by ISO/IEC 42001:2023 — record dataset hash, code commit SHA, and artifact linkage for every training run.
- NIST CSF 2.0 DE.AE maps to security-integrated drift monitoring; NIST AI RMF GOVERN maps to human oversight requirements in automated pipelines.
- Exam tip: Fully automated, human-free model deployment pipelines require explicit documentation under the EU AI Act and NIST AI RMF — not just a technical choice, but a governance decision.

10 Exam-Based MCQs — MLOps Engineer

Test yourself after reading. These questions are written in real SecAI+ exam style — scenario-heavy, with plausible distractors. Read every explanation, including for questions you answered correctly.

Q1. Scenario: An MLOps Engineer is reviewing the CI/CD pipeline for a fraud detection model. A security researcher has warned that supply chain attackers have been distributing backdoored model weights that pass standard accuracy evaluation but activate a hidden behavior when a specific input pattern (a trigger) is present. Which additional evaluation should the MLOps Engineer add to the CI/CD pipeline to detect this class of attack BEFORE production deployment?

- A) Extend the standard test set with additional examples drawn from the same training distribution
- B) Add adversarial test set evaluation that specifically includes samples designed to trigger hidden backdoor behaviors
- C) Increase the evaluation batch size to improve statistical confidence in accuracy metrics
- D) Add a fairness metric evaluation disaggregated by demographic subgroups

Answer: B **Explanation:** B is correct. Backdoored models pass standard evaluation because the trigger pattern is absent from the standard test set — the model performs normally on all non-trigger inputs. The only way to detect a backdoor through evaluation is to include adversarial test samples specifically designed to probe for hidden trigger behaviors. A (more standard test examples) increases statistical confidence in standard accuracy but still will not contain the trigger pattern. C (larger batch size) improves measurement precision but does not change what is being measured. D (fairness evaluation) is important for responsible AI but tests demographic performance gaps, not backdoor triggers.

Q2. Scenario: During a red team exercise, the MLOps team discovers that an external contractor with read access to the model training repository was also able to register a new model version directly to the production model registry by using their developer credentials, bypassing the automated training pipeline entirely. Which control would MOST effectively prevent this attack?

- A) Require human approval for all model registrations
- B) Enforce that only pipeline service accounts with signed pipeline execution proof can register model artifacts
- C) Implement input validation on the model registry API to reject oversized model files
- D) Enable audit logging on the model registry to record all registration events

Answer: B **Explanation:** B is correct. The attack requires writing a malicious artifact directly to the registry without going through the legitimate training pipeline. Restricting registration to pipeline service accounts that present proof of a completed, signed pipeline execution prevents direct artifact injection — the attacker would need to compromise the pipeline itself rather than just the registry API. A (human approval) is valuable but occurs after the artifact is submitted — it may catch suspicious artifacts but relies on humans noticing anomalies. C (file size validation) does not address the content of the artifact. D (audit logging) is a detective control that would help with incident response but does not prevent the injection.

Q3. Scenario: The MLOps monitoring dashboard shows that the production recommendation model has triggered automated retraining four times in the past 24 hours — far above the typical monthly retraining frequency. No significant business events have occurred that would explain a sudden change in user behavior at this scale. What is the MOST appropriate immediate response?

- A) Roll back the production model to the previous version and trigger a standard retraining run
- B) Disable automated retraining and investigate whether the triggering data was anomalous before promoting any retrained model
- C) Increase the retraining frequency threshold so that minor drift no longer triggers automated retraining
- D) Alert the ML Engineer that the model requires hyperparameter tuning to improve drift resilience

Answer: B **Explanation:** B is correct. An unexplained spike in retraining frequency is a potential indicator that an adversary is injecting data to force retraining cycles, possibly to get a poisoned model promoted through an automated pipeline. The correct response is to halt automated promotion and investigate the triggering data for anomalies before allowing any retrained model into production. A (roll back and retrain) may be appropriate but should not happen before understanding why retraining was triggered multiple times in one day — the retrained model may itself have been trained on poisoned data. C (raise threshold) treats the symptom, not the cause, and may cause the organization to miss legitimate degradation. D (hyperparameter tuning) addresses model stability, not the security concern of anomalous retraining frequency.

Q4. Which of the following serialization formats should MLOps Engineers enforce as a registry admission policy to prevent this class of attack?

- A) PyTorch pickle format (.pt) — the native format most widely supported by the ML ecosystem
- B) ONNX — the Open Neural Network Exchange format, which is cross-framework and widely supported
- C) SafeTensors — designed specifically to prevent arbitrary code execution during model loading
- D) HDF5 — widely used for scientific data storage and supported by TensorFlow's SavedModel format

Answer: C **Explanation:** C is correct. SafeTensors was designed by Hugging Face specifically to address the arbitrary code execution vulnerability in pickle-based model serialization. It stores only tensor data without the ability to embed executable code, making it impossible to craft a SafeTensors file that executes a payload on load. The 2024 Protect AI research specifically identified pickle-format model files (.pt, .bin) as the attack vector. B (ONNX) is a cross-framework exchange format that may contain operator definitions but has a more constrained execution model than pickle — however, SafeTensors is the format specifically designed for this security concern. A (PyTorch pickle) is explicitly the format that enables the attack. D (HDF5) is safer than pickle but is not the format specifically highlighted by the Protect AI research or the lecture as the recommended mitigation.

Q5. Scenario: An MLOps Engineer configures the drift monitoring system to route statistical anomalies in the model's input distribution to the security team's alerting channel, with context annotations

identifying whether each anomaly matches known adversarial probing patterns. Which NIST CSF 2.0 function does this monitoring capability MOST directly implement?

- A) GOVERN (GV) — establishing policies for AI risk management
- B) IDENTIFY (ID) — cataloging AI assets and risks
- C) DETECT (DE) via DE.AE — Adverse Event Analysis
- D) RESPOND (RS) — executing incident response plans

Answer: C **Explanation:** C is correct. Routing anomalous model drift patterns (unexpected input spikes, prediction distribution shifts) to the security team as potential adversarial attack indicators directly implements the DE.AE (Adverse Event Analysis) subcategory of the DETECT function — analyzing events to determine whether they constitute adverse security events. A (GOVERN) covers establishing risk management policies — not the technical detection implementation. B (IDENTIFY) covers asset and risk cataloging — a prerequisite for detection but not the detection activity itself. D (RESPOND) covers actions taken after an incident is confirmed — which follows detection but is not what drift-based security monitoring implements.

Q6. What is the PRIMARY security value of recording the Git commit SHA (rather than the branch name) in the experiment tracking system for each training run?

- A) Commit SHAs are shorter than branch names and reduce storage requirements in experiment tracking databases
- B) Commit SHAs are immutable references — the same SHA always refers to the same code, enabling reliable incident response tracing
- C) Recording commit SHAs satisfies GDPR Article 5 requirements for data minimization in AI training records
- D) Commit SHAs enable the experiment tracking system to automatically detect merge conflicts in training code

Answer: B **Explanation:** B is correct. Git commit SHAs are cryptographic hashes that immutably identify a specific state of the codebase — they cannot be changed retroactively (without rewriting history, which is detectable). Branch names, by contrast, are mutable references that can point to different commits over time. Recording commit SHAs means that months later, when investigating anomalous model behavior, an auditor can checkout the exact codebase used for a specific training run and reproduce the environment. A (storage efficiency) is incorrect — SHAs are 40-character hex strings, similar in length to many branch names. C (GDPR data minimization) is unrelated to SHA vs. branch name. D (merge conflict detection) is not a function of experiment tracking systems.

Q7. What class of attack does MITRE ATLAS v2.1 document that feature store monitoring is the primary detection mechanism for?

- A) Model extraction attacks, where adversaries systematically query the API to reconstruct a model approximation
- B) Feature-level attacks, where adversaries manipulate the feature computation process to influence model behavior

- C) Membership inference attacks, where adversaries determine whether specific data records were in the training dataset
- D) Backdoor attacks, where adversaries insert trigger patterns into training data to enable hidden model behaviors

Answer: B **Explanation:** B is correct. MITRE ATLAS v2.1 documents feature-level attacks where adversaries manipulate the feature computation process — for example, by modifying the upstream data sources that feed feature computation, or by compromising the feature store's computation logic — to influence model behavior without directly modifying model weights. Feature store monitoring detects these attacks by alerting on anomalies in feature values, feature freshness, or feature source integrity. A (model extraction) is a model API-level attack — it exploits the inference endpoint, not the feature store. C (membership inference) exploits model outputs to infer training data membership — not a feature store attack. D (backdoor attacks) are typically implemented via training data poisoning, not feature store manipulation specifically.

Q8. Which of the following BEST describes why ISO/IEC 42001:2023 is relevant to MLOps experiment tracking practices?

- A) ISO/IEC 42001:2023 specifies the technical format for experiment tracking records that all AI management systems must use
- B) ISO/IEC 42001:2023 requires organizations to maintain records sufficient to enable AI system behavior to be explained and audited — experiment tracking provides this traceability chain
- C) ISO/IEC 42001:2023 prohibits automated retraining pipelines without ISO-certified human oversight committees
- D) ISO/IEC 42001:2023 mandates a minimum experiment tracking tool from a list of approved vendors

Answer: B **Explanation:** B is correct. ISO/IEC 42001:2023 (AI Management System) requires organizations to establish processes that enable AI system behavior to be explained and audited — but does not prescribe specific technical formats or tools. Experiment tracking systems that record dataset version, code version, hyperparameters, and evaluation metrics provide the traceability chain that satisfies this requirement. A (specific technical format) is incorrect — ISO/IEC 42001 is a management system standard, not a technical format specification. C (prohibiting automated retraining) is incorrect — the standard sets governance requirements but does not prohibit automation; it requires oversight to be documented. D (approved vendor list) is incorrect — ISO management system standards are vendor-neutral.

Q9. Which of the following BEST explains the security concern with pipeline orchestration interfaces such as the Airflow web UI?

- A) Airflow stores pipeline execution logs in plaintext, which may contain sensitive training data
- B) An attacker with access to the Airflow UI can modify pipeline DAGs to inject malicious steps, redirect artifact destinations, or trigger pipelines with attacker-controlled parameters
- C) Airflow does not support TLS for web UI connections, making credentials vulnerable to network interception

D) The Airflow scheduler runs as root, so a compromised Airflow instance automatically has host-level access

Answer: B **Explanation:** B is correct. The Airflow web UI provides the ability to view, modify, and trigger pipeline DAGs. An attacker with UI access can modify a training pipeline to inject a step that exfiltrates training data, insert a model weight replacement step before artifact registration, trigger a pipeline with attacker-controlled parameters, or redirect model artifact destinations to an attacker-controlled registry. This pipeline-level access is more powerful than individual model or data compromise because it affects all models processed through the pipeline. A (plaintext logs) is a valid concern in some configurations but is not the primary security concern described. C (Airflow lacks TLS) is incorrect — Airflow supports TLS; misconfigured deployments may not enforce it, but this is not inherent to Airflow. D (Airflow scheduler runs as root) is not universally true and not the primary concern described in the lecture.

Q10. Under which regulatory requirement must organizations explicitly document the decision to eliminate human oversight from automated model deployment pipelines?

- A) GDPR Article 22 — Automated individual decision-making, including profiling
- B) EU AI Act (2024) and NIST AI RMF 1.0, which both require human oversight decisions to be explicitly documented and risk-assessed
- C) HIPAA Security Rule, which requires human review of all automated decisions affecting protected health information
- D) PCI DSS v4.0, which requires human approval for all automated changes to systems handling cardholder data

Answer: B **Explanation:** B is correct. The lecture specifically cites the EU AI Act (2024) and NIST AI RMF 1.0 as requiring that decisions to eliminate human oversight from consequential AI operations — such as fully automated model deployment without a human approval gate — be explicitly documented and risk-assessed. These frameworks apply specifically to AI system governance. A (GDPR Article 22) covers automated decision-making affecting individuals in specific legal or similarly significant contexts — relevant to AI-generated decisions, but not specifically to the MLOps pipeline governance decision described. C (HIPAA Security Rule) does not specifically require human review of automated model deployment decisions. D (PCI DSS v4.0) applies to cardholder data environment changes — relevant in financial contexts but not the general requirement described in the lecture.